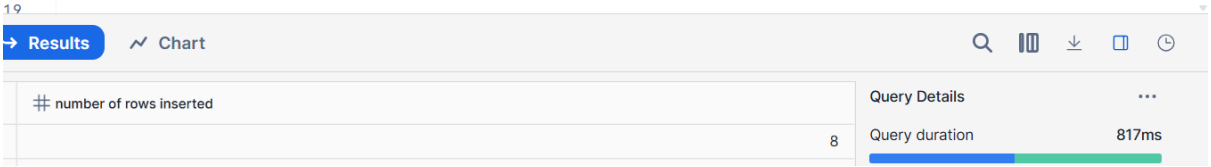
Part 1: Dynamic Tables – Core Practice

1. Create a source table orders_src with:
    - order_id
    - order_date
    - region
    - product
    - amount

```
create database management;
CREATE OR REPLACE TABLE orders_src (
    order_id INT,
    order_date DATE,
    region STRING,
    product STRING,
    amount NUMBER(10,2)
);
```

2. Insert at least 8 records covering multiple regions and products.

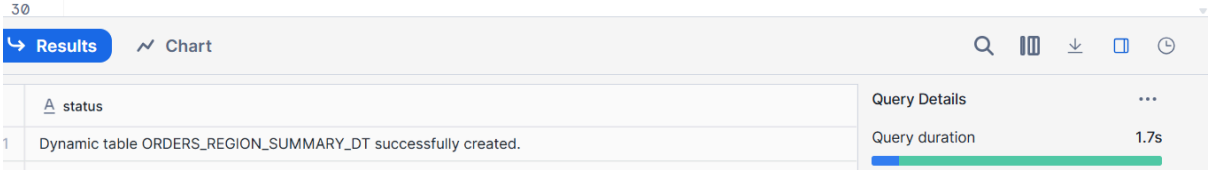
```
INSERT INTO orders_src VALUES
(1, '2025-01-01', 'North', 'Laptop', 50000),
(2, '2025-01-02', 'South', 'Mobile', 20000),
(3, '2025-01-03', 'East', 'Tablet', 15000),
(4, '2025-01-04', 'West', 'Laptop', 52000),
(5, '2025-01-05', 'North', 'Mobile', 22000),
(6, '2025-01-06', 'South', 'Tablet', 18000),
(7, '2025-01-07', 'East', 'Laptop', 48000),
(8, '2025-01-08', 'West', 'Mobile', 21000);
```



3.Create a Dynamic Table orders_region_summary_dt that shows:

- region
- total_orders
- Total_sales Set TARGET_LAG = 5 MINUTE.

```
CREATE OR REPLACE DYNAMIC TABLE orders_region_summary_dt
TARGET_LAG = '5 MINUTE'
WAREHOUSE = COMPUTE_WH
AS
SELECT
    region,
    COUNT(order_id) AS total_orders,
    SUM(amount) AS total_sales
FROM orders_src
GROUP BY region;
```



Initial Check :

```
SELECT * FROM orders_region_summary_dt;
```

Results | Chart

	REGION	TOTAL_ORDERS	TOTAL_SALES
1	East	2	63000.00
2	North	2	72000.00
3	South	2	38000.00
4	West	2	73000.00

Query Details
Query duration 178ms
Rows 4
Query ID 01c1bfb3-0001-7de7-00...
Show more

4. Insert new records into orders_src and verify that the Dynamic Table updates automatically.

```
INSERT INTO orders_src VALUES
(9, '2025-01-09', 'North', 'Tablet', 17000),
(10, '2025-01-10', 'South', 'Laptop', 55000);
```

Results | Chart

	number of rows inserted
1	2

Query Details
Query duration 259ms

Wait **up to 5 minutes**, then:

```
SELECT * FROM orders_region_summary_dt;
```

Results | Chart

	REGION	TOTAL_ORDERS	TOTAL_SALES
1	North	3	89000.00
2	South	3	93000.00
3	East	2	63000.00
4	West	2	73000.00

Query Details
Query duration 133ms
Rows 4
Query ID 01c1bfb7-0001-7de7-00...
Show more

Final result :

- North total_orders increases
- South total_orders & total_sales increase
- No manual refresh needed → Dynamic Table auto-updates

5. Update the amount for an existing order and observe the Dynamic Table behavior.

```
UPDATE orders_src
SET amount = 60000
WHERE order_id = 1;
```

Results | Chart

	number of rows updated	number of multi-joined rows updated
1	1	0

Query Details
Query duration 288ms

After TARGET_LAG:

```
38  SELECT * FROM orders_region_summary_dt;
```

Results | Chart

	REGION	TOTAL_ORDERS	TOTAL_SALES
1	North	3	89000.00
2	South	3	93000.00
3	East	2	63000.00
4	West	2	73000.00

Query Details
Query duration 58ms
Rows 4
Query ID 01c1bfbc-0001-7de7-00...
Show more

Final result :

- **North total_sales increases**
- Order count remains unchanged
- Dynamic Table reflects the update automatically

6. Delete one record from orders_src.
Re-query the Dynamic Table and note the result.

```
44  DELETE FROM orders_src
45  WHERE order_id = 2;
```

Results | Chart

	number of rows deleted
1	1

Query Details
Query duration 264ms

Re-query after lag :

```
38  SELECT * FROM orders_region_summary_dt;
39
```

Results | Chart

	REGION	TOTAL_ORDERS	TOTAL_SALES
1	East	2	63000.00
2	West	2	73000.00
3	North	3	99000.00
4	South	3	93000.00

Query Details
Query duration 131ms
Rows 4
Query ID 01c1bfbe-0001-7de7-00...
Show more

Final result :

- South total_orders decreases by 1
- South total_sales decreases
- Dynamic Table reflects delete automatically

7. Suspend the warehouse used by the Dynamic Table.
Query the Dynamic Table and observe what happens.

```
47  ALTER WAREHOUSE COMPUTE_WH SUSPEND;
```

Results | Chart

	status
1	Statement executed successfully.

Query Details
Query duration 68ms

Now query:

```
38  SELECT * FROM orders_region_summary_dt;
39
```

Results | Chart

	REGION	TOTAL_ORDERS	TOTAL_SALES
1	North	3	99000.00
2	East	2	63000.00
3	West	2	73000.00
4	South	2	73000.00

Query Details
Query duration 574ms
Rows 4
Query ID 01c1bfc1-0001-7de7-00...
Show more

Final result :

- Query **fails or returns stale data**
- Dynamic Table **cannot refresh** without an active warehouse
- Once warehouse resumes, it refreshes automatically

```
49  ALTER WAREHOUSE COMPUTE_WH RESUME;
50
```

Results | Chart

	status
1	Statement executed successfully.

Query Details
Query duration 54ms

Part 2: Dynamic Table Dependency Chain

8. Create a Dynamic Table orders_clean_dt that:
    - Filters out records where amount <= 0.

```
51  CREATE OR REPLACE DYNAMIC TABLE orders_clean_dt
52  TARGET_LAG = '1 MINUTE'
53  WAREHOUSE = compute_wh
54  AS
55  SELECT
56      order_id,
57      order_date,
58      region,
59      product,
60      amount
61  FROM orders_src
62  WHERE amount > 0;
```

Results | Chart

	status
1	Dynamic table ORDERS_CLEAN_DT successfully created.

Query Details
Query duration 833ms

9. Create another Dynamic Table orders_product_kpi_dt that:
    - Depends on orders_clean_dt
    - Shows total_sales and total_orders by product.

This Dynamic Table **depends on orders_clean_dt**, not the base table.

```
CREATE OR REPLACE DYNAMIC TABLE orders_product_kpi_dt
TARGET_LAG = '1 MINUTE'
WAREHOUSE = compute_wh
AS
SELECT
    product,
    COUNT(order_id) AS total_orders,
    SUM(amount) AS total_sales
FROM orders_clean_dt
GROUP BY product;
```

Results | Chart

status
Dynamic table ORDERS_PRODUCT_KPI_DT successfully created.

Query Details

Query duration 1.2s

Snowflake automatically understands the dependency chain:

```
nginx

orders_src
    ↓
orders_clean_dt
    ↓
orders_product_kpi_dt
```

No manual scheduling needed ✅

10. Insert invalid and valid records into orders_src.
    Verify how both Dynamic Tables behave.

```
INSERT INTO orders_src VALUES
(201, '2024-01-10', 'East', 'Laptop', 1200),
(202, '2024-01-11', 'West', 'Mobile', -500),   -- ❌ Invalid
(203, '2024-01-12', 'South', 'Tablet', 0),     -- ❌ Invalid
(204, '2024-01-13', 'North', 'Laptop', 1500),  -- ✅ Valid
(205, '2024-01-14', 'East', 'Mobile', 800);    -- ✅ Valid
```

Results | Chart

	number of rows inserted
1	5

Query Details

Query duration 257ms

```
SELECT * FROM orders_clean_dt;
```

Results | Chart

	ORDER_ID	ORDER_DATE	REGION	PRODUCT	AMOUNT
1	1	2025-01-01	North	Laptop	60000.00
2	3	2025-01-03	East	Tablet	15000.00
3	4	2025-01-04	West	Laptop	52000.00
4	5	2025-01-05	North	Mobile	22000.00
5	6	2025-01-06	South	Tablet	18000.00
6	7	2025-01-07	East	Laptop	48000.00
7	8	2025-01-08	West	Mobile	21000.00
8	9	2025-01-09	North	Tablet	17000.00

Query Details
Query duration 59ms
Rows 15
Query ID 01c1bfdb-0001-7de7-00...
Show more
ORDER_ID

Only records with **amount > 0** appear.

```
SELECT * FROM orders_product_kpi_dt;
```

Results | Chart

	PRODUCT	TOTAL_ORDERS	TOTAL_SALES
1	Mobile	4	44600.00
2	Laptop	8	221200.00
3	Tablet	3	50000.00

Query Details
Query duration 143ms
Rows 3
Query ID 01c1bfdc-0001-7de7-00...

KPIs calculated **only from clean data**.

11. Change data in the base table and confirm refresh order without manual scheduling.

```
UPDATE orders_src
SET amount = 2000
WHERE order_id = 201;
```

Results | Chart

	number of rows updated	number of multi-joined rows updated
1	2	0

Query Details
Query duration 287ms

- orders_clean_dt refreshes first
- orders_product_kpi_dt refreshes next
- No manual task, no cron, no trigger

Part 3: SCD Type 1 Using Dynamic Tables

12. Create a source table customer_src with:

- customer_id
- customer_name
- city
- updated_at

```sql
CREATE OR REPLACE TABLE customer_src (
    customer_id   INT,
    customer_name STRING,
    city          STRING,
    updated_at    TIMESTAMP
);
```

Results | Chart

status
Table CUSTOMER_SRC successfully created.

Query Details
Query duration 145ms

13. Insert multiple records per customer with different timestamps.

```sql
INSERT INTO customer_src VALUES
(1, 'Ravi',   'Chennai',   '2024-01-01 10:00:00'),
(1, 'Ravi',   'Bangalore','2024-01-05 12:00:00'),
(2, 'Anita', 'Delhi',     '2024-01-02 09:30:00'),
(2, 'Anita', 'Mumbai',   '2024-01-06 15:45:00'),
(3, 'John',   'Hyderabad','2024-01-03 11:15:00');
```

Results | Chart

number of rows inserted
5

Query Details
Query duration 294ms

- Same customer appears multiple times
- Each row represents a **new version**

14. Create a Dynamic Table customer_dim_scd1_dt that keeps only the latest record per customer.

Answer :

- Keeps **only the latest record per customer**
- Older versions are **automatically removed**

```sql
CREATE OR REPLACE DYNAMIC TABLE customer_dim_scd1_dt
TARGET_LAG = '1 MINUTE'
WAREHOUSE = compute_wh
AS
SELECT
    customer_id,
    customer_name,
    city,
    updated_at
FROM customer_src
QUALIFY ROW_NUMBER() OVER (
    PARTITION BY customer_id
    ORDER BY updated_at DESC
) = 1;
```

Results | Chart

status
Dynamic table CUSTOMER_DIM_SCD1_DT successfully created.

Query Details
Query duration 812ms

Why this works (SCD Type 1 logic)

- ROW_NUMBER() ranks rows per customer
- Latest timestamp gets rank = 1
- Dynamic Table always shows current version only

15. Insert a new version of an existing customer and verify that old data disappears.

```
INSERT INTO customer_src VALUES
(1, 'Ravi', 'Pune', '2024-01-10 09:00:00');
```

Results | Chart

number of rows inserted
1

Query Details
Query duration 399ms

Check Dynamic Table

```
SELECT *
FROM customer_dim_scd1_dt
WHERE customer_id = 1;
```

Results | Chart

CUSTOMER_ID	CUSTOMER_NAME	CITY	UPDATED_AT
1	Ravi	Pune	2024-01-10 09:00:00.000

Query Details
Query duration 145ms

- Old cities (Chennai, Bangalore) are gone
- Only **latest data exists**

Final result :

- SCD Type 1 with Dynamic Tables means “keep only the latest customer details and forget the past.”
- Snowflake does this automatically whenever new data arrives.

Part 4: Stored Procedures – ETL Practice

16. Create a target table orders_audit.

```
CREATE OR REPLACE TABLE orders_audit (
    audit_id        INT AUTOINCREMENT,
    load_time       TIMESTAMP,
    record_count    INT
);
```

Results | Chart

status
Table ORDERS_AUDIT successfully created.

Query Details
Query duration 143ms
Rows 1

17. Write a Stored Procedure that:

- Inserts rows into orders_audit
- Captures load timestamp
- Captures record count

```sql
CREATE OR REPLACE PROCEDURE sp_load_orders_audit()
RETURNS STRING
LANGUAGE SQL
AS
$$
DECLARE
    v_count INT;
BEGIN
    -- Count records from source
    SELECT COUNT(*) INTO :v_count FROM orders_src;

    -- Insert audit record
    INSERT INTO orders_audit (load_time, record_count)
    VALUES (CURRENT_TIMESTAMP, v_count);

    RETURN 'Audit load successful';
END;
$$;
```

Results | Chart

status
Function SP_LOAD_ORDERS_AUDIT successfully created.

Query Details — Query duration 53ms

- Captures **load timestamp**
- Captures **record count**

18. Execute the Stored Procedure manually.

```sql
CREATE OR REPLACE PROCEDURE sp_load_orders_audit()
RETURNS STRING
LANGUAGE SQL
EXECUTE AS OWNER
AS
$$
DECLARE
    v_count INTEGER;
BEGIN
    -- Get record count
    v_count := (SELECT COUNT(*) FROM orders_src);

    -- Insert audit record
    INSERT INTO orders_audit (load_ts, record_count)
    VALUES (CURRENT_TIMESTAMP(), v_count);

    RETURN 'SUCCESS: Orders audit loaded. Records = ' || v_count;

EXCEPTION
    WHEN OTHER THEN
        RETURN 'ERROR: ' || SQLERRM;
END;
```

Results | Chart

status
Function SP_LOAD_ORDERS_AUDIT successfully created.

Query Details — Query duration 62ms

```sql
SHOW PROCEDURES LIKE 'SP_LOAD_ORDERS_AUDIT';
CALL sp_load_orders_audit();
```

Results | Chart

created_on	name	schema_name	is_builtin	is_aggregate	is_ansi	min_num_arguments	max_num_argum
2026-01-15 07:04:43	SP_LOAD_ORDERS_AUI	PUBLIC	N	N	N	0	

Query Details — Query duration 29ms

19. Schedule the Stored Procedure using a Task.

Create task : Runs **every 1 hour**

```sql
CREATE OR REPLACE TASK task_load_orders_audit
WAREHOUSE = compute_wh
SCHEDULE = 'USING CRON 0 */1 * * * UTC'
AS
CALL sp_load_orders_audit();
```

Results | Chart

status
Task TASK_LOAD_ORDERS_AUDIT successfully created.

Query Details — Query duration 61ms

Enable task :

```
188  ALTER TASK task_load_orders_audit RESUME;
189
```

Results | Chart

status
Statement executed successfully.

Query Details — Query duration 82ms

Verify task :

```
190  SHOW TASKS;
191
```

Results | Chart

created_on	name	id	database_name	schema_name	owner	comment	warehouse	schedule
2026-01-15 07:11:32	TASK_LOAD_ORDERS_A	01c1c00f-78e8-a6c7-	MANAGEMENT	PUBLIC	ACCOUNTADMIN		COMPUTE_WH	USING CRON 0 */1 * *

Query Details — Query duration 31ms

Part 5: Stored Procedures vs Dynamic Tables

Feature	Stored Procedure	Dynamic Table
UPDATE / DELETE	✅ Yes	❌ No
Conditional logic	✅ Yes	❌ Limited
Modifies source table	✅ Yes	❌ No
Auto refresh	❌ (Task needed)	✅ Built-in
Best for	ETL, DML, business rules	Reporting, transformations

21. Use a Stored Procedure to:

- Update a flag column in a table conditionally.

1️⃣ Create source table

```
192  CREATE OR REPLACE TABLE orders_flag (
193      order_id INT,
194      amount NUMBER,
195      high_value_flag STRING
196  );
197
```

Results | Chart

status
Table ORDERS_FLAG successfully created.

Query Details — Query duration 135ms

2️⃣ Insert sample data

```
199  INSERT INTO orders_flag VALUES
200  (1, 500, NULL),
201  (2, 1500, NULL),
202  (3, 3000, NULL);
203
204
```

Results | Chart

number of rows inserted
3

Query Details — Query duration 314ms

3️⃣ Stored Procedure to update flag conditionally

```sql
CREATE OR REPLACE PROCEDURE sp_update_high_value_flag()
RETURNS STRING
LANGUAGE SQL
AS
$$
BEGIN
    UPDATE orders_flag
    SET high_value_flag =
        CASE
            WHEN amount > 1000 THEN 'Y'
            ELSE 'N'
        END;

    RETURN 'Flag updated successfully';
END;
$$;
```

Results | Chart

status
Function SP_UPDATE_HIGH_VALUE_FLAG successfully created.

Query Details — Query duration 45ms

4 Execute procedure

```sql
CALL sp_update_high_value_flag();
```

Results | Chart

SP_UPDATE_HIGH_VALUE_FLAG
Flag updated successfully

Query Details — Query duration 462ms

5 Verify

```sql
SELECT * FROM orders_flag;
```

Results | Chart

	ORDER_ID	AMOUNT	HIGH_VALUE_FLAG
1	1	500	N
2	2	1500	Y
3	3	3000	Y

Query Details — Query duration 136ms — Rows 3 — Query ID 01c1c046-0001-7de7-0...

22. Try implementing the same logic using a Dynamic Table.
    Note the limitation.

```sql
CREATE OR REPLACE DYNAMIC TABLE orders_flag_dt
TARGET_LAG = '1 minute'
WAREHOUSE = compute_wh
AS
SELECT
    order_id,
    amount,
    CASE
        WHEN amount > 1000 THEN 'Y'
        ELSE 'N'
    END AS high_value_flag
FROM orders_flag;
```

Results | Chart

status
Dynamic table ORDERS_FLAG_DT successfully created.

Query Details — Query duration 911ms

This creates a table . But does **not update** orders_flag.

Why Dynamic Table FAILS for this requirement

- **Dynamic Tables are READ-ONLY derived objects**

They are :

- Cannot UPDATE existing tables
- Cannot modify source data
- Cannot run procedural logic

- Cannot execute conditional DML

They only:

- SELECT
- Transform
- Auto-refresh results

Part 6: Dashboard-Ready Queries

23. Write a query on orders_region_summary_dt for:
- Total sales by region (descending).

```
SELECT
    region,
    SUM(total_sales) AS total_sales
FROM orders_region_summary_dt
GROUP BY region
ORDER BY total_sales DESC;
```

Results | Chart

REGION	TOTAL_SALES
North	102000.00
South	73000.00
West	72000.00
East	68600.00

Query Details
Query duration 669ms
Rows 4
Query ID 01c1c054-0001-7de7-0...

24. Write a query on orders_product_kpi_dt for:
- Top 3 products by sales.

```
SELECT
    product,
    SUM(total_sales) AS total_sales
FROM orders_product_kpi_dt
GROUP BY product
ORDER BY total_sales DESC
LIMIT 3;
```

Results | Chart

	PRODUCT	TOTAL_SALES
1	Laptop	222000.00
2	Tablet	50000.00
3	Mobile	44600.00

Query Details
Query duration 259ms
Rows 3

25. Filter dashboard data for:
- Region = 'East'

```
SELECT *
FROM orders_src
WHERE region = 'East'
```

Results | Chart

ORDER_ID	ORDER_DATE	REGION	PRODUCT	AMOUNT
3	2025-01-03	East	Tablet	15000.00
7	2025-01-07	East	Laptop	48000.00
205	2024-01-14	East	Mobile	800.00
201	2024-01-10	East	Laptop	2000.00

Query Details
Query duration 26ms
Rows 6
Query ID 01c1c064-0001-7de7-0...

- Product = 'Laptop'

```
SELECT *
FROM orders_product_kpi_dt
WHERE product = 'Laptop';
```

Results | Chart

PRODUCT	TOTAL_ORDERS	TOTAL_SALES
Laptop	8	222000.00

Query Details
Query duration 48ms

Part 7: Monitoring & Operations

27. Change TARGET_LAG to a higher value and observe refresh behavior.

```
ALTER DYNAMIC TABLE orders_region_summary_dt
SET TARGET_LAG = '30 minutes';
```

Results | Chart

status
Statement executed successfully.

Query Details
Query duration 53ms

- Refresh happens **less frequently**
- **Lower compute cost**
- Data may be **slightly less fresh**
- Good for **dashboards**, not real-time analytics

28. Separate compute by assigning:

- One warehouse for ETL

```
CREATE OR REPLACE WAREHOUSE etl_wh
WAREHOUSE_SIZE = MEDIUM
AUTO_SUSPEND = 60
AUTO_RESUME = TRUE;
```

Results | Chart

status
Warehouse ETL_WH successfully created.

Query Details
Query duration 93ms

- One warehouse for Dynamic Tables

```
CREATE OR REPLACE WAREHOUSE dt_wh
WAREHOUSE_SIZE = SMALL
AUTO_SUSPEND = 60
AUTO_RESUME = TRUE;
```

Results | Chart

status
Warehouse DT_WH successfully created.

Query Details
Query duration 76ms

Assign warehouse to Dynamic Table :

```
ALTER DYNAMIC TABLE orders_region_summary_dt
SET WAREHOUSE = dt_wh;
```

Results | Chart

status
Statement executed successfully.

Query Details
Query duration 45ms

✅ Why this separation is important

Purpose	Benefit
ETL warehouse	Heavy transformations won't affect dashboards
Dynamic Table warehouse	Predictable refresh & stable performance
Separate cost tracking	Easy billing & optimization

Part 8: Failure & Recovery Practice

29. Introduce invalid SQL in a Dynamic Table definition and observe behavior.

1️⃣ Introduce Invalid SQL in a Dynamic Table (Example: Create a Dynamic Table (working version)

```
CREATE OR REPLACE DYNAMIC TABLE orders_summary_dt
TARGET_LAG = '1 MINUTE'
WAREHOUSE = compute_wh
AS
SELECT
    region,
    COUNT(*) AS total_orders,
    SUM(amount) AS total_sales
FROM orders_src
GROUP BY region;
```

Results | Chart

status
Dynamic table ORDERS_SUMMARY_DT successfully created.

Query Details — Query duration 1.7s

❌ Introduce an Error (Invalid Column)

Modify the Dynamic Table with an invalid column:

```
CREATE OR REPLACE DYNAMIC TABLE orders_summary_dt
TARGET_LAG = '1 MINUTE'
WAREHOUSE = compute_wh
AS
SELECT
    region,
    COUNT(*) AS total_orders,
    SUM(invalid_amount) AS total_sales   -- ❌ column does not exist
FROM orders_src
GROUP BY region;
```

Results | Chart

⚠ Error: invalid identifier 'INVALID_AMOUNT' (line 394)

Query Details — Query duration 37ms — Rows 0 — Query ID 01c1c324-0001-7eb2-0...

Observe Behavior:

```
DESCRIBE DYNAMIC TABLE orders_summary_dt;
```

Results | Chart

name	type	kind	null?	default	primary key	unique key	check	expression	comment	policy name
REGION	VARCHAR(16777216)	COLUMN	Y	null	N	N	null	null	null	null
TOTAL_ORDER	NUMBER(18,0)	COLUMN	Y	null	N	N	null	null	null	null
TOTAL_SALES	NUMBER(22,2)	COLUMN	Y	null	N	N	null	null	null	null

Query Details — Query duration 69ms — Rows 3 — Query ID 01c1c324-0001-7e8f-00...

What happens:

- Dynamic Table **enters FAILED state**
- Data **does NOT refresh**
- Error message shows **invalid column**
- Snowflake does **not drop** the Dynamic Table
- Last successful data (if any) remains available

30. Fix the issue and confirm auto-recovery.

```
CREATE OR REPLACE DYNAMIC TABLE orders_summary_dt
TARGET_LAG = '1 MINUTE'
WAREHOUSE = compute_wh
AS
SELECT
    region,
    COUNT(*) AS total_orders,
    SUM(amount) AS total_sales   -- ✅ fixed
FROM orders_src
GROUP BY region;
```

Results | Chart

status
Dynamic table ORDERS_SUMMARY_DT successfully created.

Query Details
Query duration 1.4s

Auto-Recovery Behavior :

- Dynamic Table **automatically resumes refresh**
- Status changes from **FAILED → ACTIVE**
- No manual restart required

31. Stop the Task running the Stored Procedure and restart it.

```
CREATE OR REPLACE TASK orders_audit_task
WAREHOUSE = compute_wh
SCHEDULE = '5 MINUTE'
AS
CALL load_orders_audit();
```

Results | Chart

status
Task ORDERS_AUDIT_TASK successfully created.

Query Details
Query duration 121ms

Suspend the Task

```
ALTER TASK orders_audit_task SUSPEND;
```

Results | Chart

status
Statement executed successfully.

Query Details
Query duration 60ms

- Task **stops executing**
- stored Procedure **will not run**
- No new audit records inserted

Restart the Task :

```
ALTER TASK orders_audit_task RESUME;
```

Results | Chart

	status
1	Statement executed successfully.

Query Details
Query duration 240ms
Rows 1

- Task resumes as per schedule
- Stored Procedure executes again
- Data loading continues normally

Simple Hierarchy :

Feature	Behavior
Dynamic Table failure	Goes to FAILED state
Data loss	❌ No
Fix SQL	Auto-recovers
Manual restart	❌ Not required
Task suspend	Stops execution
Task resume	Continues scheduling